

Turning Ninety Degrees

To write an autonomous command that gets the ROMI to do a *precise* turn of 90 degrees you should first make sure that you can track what is going on with the turn. One method is to use the gyro.

Turning with the Gyro.

Calibrate the Gyro

Romi's gyro is not great, and it will be helpful if you calibrate the ROMI's gyro before doing anything else. The gyro is known to drift (give different, generally increasing or decreasing, values over time). You can calibrate the gyro by opening a webpage with address 10.0.0.2 while on the ROMI's wifi. Then choose the ROMI option on the side. Change the ROMI to **writable** by choosing the button at top center of the screen. Then scroll down to the option to calibrate. It takes 75 seconds.

Send information to network tables

You will want to see what the gyro is saying. To do this go the **periodic** method in `drivetrain.py` and add these lines.

```
self.nt_drivetrain.putNumber("Z Angle", self.getGyroAngleZ()*180/math.pi)
self.nt_drivetrain.putNumber("Z Angle Goal", 90)
```

and at the top of the `drivetrain.py` and add:

```
import math
```

Adding a plot When you start the simulation now you can use the menu item called **plot** to add a plot display. Hit the button to **start a new plot**, and then drag over the **Z angle** and **Z angle Goal**. You can zoom and pan the display to make it between 0 and 90. Test it in teleoperated mode.

Create the command

Now that you can see what is happening with the robot you ready to make the command. It is important to remember that the units for the turn is in radians so our goal is going to be $\pi/2$ radians.

Create a new file `turnninety.py`. Import the `Command` class from `commands2` library and the `Drivetrain` from your `subsystem.drivetrain`.

We want to use `wpilib`'s `PIDController` class. Import that from `wpimath.controller`.

Create a subclass of `Command` called `TurnNinety`. It will have methods:

- `__init__` : Here you will bring in the drivetrain, make it a requirement, create a `PIDController`. You will have to choose values for the k_p , k_i , and k_d and then tune them later. Using what you know about the units in this situation pick a k_p . The value for k_i is always 0, and the k_d will be tuned much later, so for now set it to 0.
- `initialize`: Here you will reset the Gyro, give the pidcontroller a setpoint and a tolerance.
- `execute`: Use the `PIDController`'s calculate function based on the current reading of the Gyro. Then give the result of that calculation to the turn part of `arcadeDrive`.
- `isFinished`: should return what the `PIDController` thinks about being at its setpoint, it has a method just for this.
- `end`: should stop the motion of the robot.

Testing and Tuning

Load the code and find any errors. Here are things to watch for...

- The robot never stops turning (and in fact seems to go faster and faster)... Possibly your `PIDController`'s calculate method is giving you the opposite of what you want.
- The robot doesn't make a complete 90 degree turn, then k_p is too small

- The robot goes past 90 degrees, then comes back a little but not enough. This means that you have enough k_p , and now need to add some k_d to make it not have as much error. Start with number around .01 for k_d then try doubling or tripling every time, or if seems too big cut it in half or a third.

Adding a Chooser for Autonomous Commands (With Sequential Commands)

It is useful to have a `SendableChooser` to let you pick between autonomous commands on the dashboard.

Why Use a Chooser?

During matches or testing, you might want to switch between different autonomous routines like driving straight, turning, or a combination of actions. A `SendableChooser` lets you select the desired command from the driver station.

Set Up a Chooser for Autonomous Commands

Step 1: Import Required Classes Add the following to the top of your `robot.py`:

```
from wpilib import SmartDashboard, SendableChooser
from autonomous.drivestraight import DriveStraight
from autonomous.turnninety import TurnNinety
```

Step 2: Create and Configure the Chooser Inside your `robotInit()` method:

```
self.chooser = SendableChooser()

# Individual commands
self.chooser.setDefaultOption("Drive Straight", DriveStraight(self.drivetrain))
self.chooser.addOption("Turn Ninety", TurnNinety(self.drivetrain))

# Sequential command group: drive straight, then turn
self.chooser.addOption("Drive + Turn",
    SequentialCommandGroup(
        DriveStraight(self.drivetrain),
        TurnNinety(self.drivetrain)
    )
)

SmartDashboard.putData("Autonomous Mode", self.chooser)
```

Step 3: Run the Selected Autonomous Command Update your `autonomousInit()`:

```
def autonomousInit(self):
    self.autoCommand = self.chooser.getSelected()
    if self.autoCommand:
        self.autoCommand.schedule()
```

Suggested File Structure

```
autonomous/
    drivestraight.py
    turnninety.py
```

Each file should define a class that inherits from `Command`.

Import from autonomous directory instead of particular files

Currently we are telling python which precise file each autonomous is contained in. We can use the directory instead if in the directory we add a file called `__init__.py` with instructions to import the desired classes.

```
from .drivestraight import DriveStraight
from .turnninety import TurnNinety

__all__ = ["DriveStraight", "TurnNinety"]
```

The `__all__` list defines what gets imported when someone does `from autonomous import *`.

Testing and Using the Chooser

1. In the Robot Simulation window there is a Network Tables Menu, if you go to the SmartDashboard option you can choose “Autonomous Mode”, and a small window will appear where you can make your choice.
 2. Look for the “Autonomous Mode” chooser.
 3. Select between:
 - “Drive Straight”
 - “Turn Ninety”
 4. Enable Autonomous mode to see the selected routine in action.
-

Bonus: Create More Complex Sequences

You can chain as many commands as you want using `SequentialCommandGroup`:

```
class DriveSquare(SequentialCommandGroup):
    def __init__(self, drivetrain):
        super().__init__(
            DriveStraight(drivetrain),
            TurnNinety(drivetrain),
            DriveStraight(drivetrain),
            TurnNinety(drivetrain),
            DriveStraight(drivetrain),
            TurnNinety(drivetrain),
            DriveStraight(drivetrain),
            TurnNinety(drivetrain)
        )
```